



Java Programming

Operators & If-Else Conditions

Teaching Guide with Problem-Solving Approach

Section 1 — Java Operators

Operators are the building blocks of logic. Before diving into syntax, let's build intuition.

1.1 Arithmetic Operators

These work exactly like math class — plus, minus, multiply, divide, and one new one:

Operator	Meaning & Example
$a + b$	Addition: $5 + 3 = 8$
$a - b$	Subtraction: $10 - 4 = 6$
$a * b$	Multiplication: $6 * 7 = 42$
a / b	Division: $10 / 4 = 2$ (integer division!)
$a \% b$	Modulus (remainder): $10 \% 3 = 1$

Integer Division Trap!

```
int result = 10 / 4;    // result = 2, NOT 2.5 !
```

```
double result = 10.0 / 4; // result = 2.5 ✓
```

When both operands are integers, Java discards the decimal. Always use double if you need fractions.

Example — Calculating a pizza share:

```
int totalSlices = 8;
int people = 3;

int each = totalSlices / people;    // 2 slices each
int leftover = totalSlices % people; // 2 slices remaining

System.out.println("Each person gets: " + each);
System.out.println("Leftover slices: " + leftover);
```

1.2 Comparison (Relational) Operators

Comparison operators always return a boolean value — either true or false. They are the questions your program asks.

Operator	Meaning & Example (assume a=10, b=5)
a == b	Equal to? → false
a != b	Not equal to? → true
a > b	Greater than? → true
a < b	Less than? → false
a >= b	Greater than or equal? → true
a <= b	Less than or equal? → false

⊗ = vs == (Most Common Beginner Mistake)

```
int age = 18;           // = is ASSIGNMENT (storing a value)
if (age == 18) { }      // == is COMPARISON (asking a question)
Think of == as asking: 'is age the same as 18?'
```

1.3 Logical Operators

Logical operators combine multiple conditions. Think of them as the words AND, OR, NOT in plain English.

Operator	Meaning
&& (AND)	Both conditions must be true
(OR)	At least one condition must be true
! (NOT)	Reverses the boolean value

Real-world analogy:

```
// You can ride the roller coaster if: height >= 140 AND age >= 12
boolean canRide = (height >= 140) && (age >= 12);

// You get a discount if: student OR senior
boolean getsDiscount = isStudent || isSenior;

// Door is unlocked if it's NOT locked
boolean doorOpen = !isDoorLocked;
```

1.4 Increment / Decrement & Compound Assignment

Shorthand	Equivalent to
i++ or ++i	i = i + 1

<code>i--</code> or <code>--i</code>	<code>i = i - 1</code>
<code>i += 5</code>	<code>i = i + 5</code>
<code>i -= 3</code>	<code>i = i - 3</code>
<code>i *= 2</code>	<code>i = i * 2</code>
<code>i /= 4</code>	<code>i = i / 4</code>

i++ vs ++i — The Subtle Difference:

```
int i = 5; int a = i++; // a = 5, THEN i becomes 6 (post-increment)
int i = 5; int b = ++i; // i becomes 6 FIRST, then b = 6 (pre-increment)
```

In a simple `i++` statement on its own line, there's no difference. It only matters when used inside an expression.

Section 2 — The Problem-Solving Mindset

Syntax is the easy part. The real skill in programming is learning to think like a computer — breaking down problems into clear decisions before touching the keyboard.



The 4-Step Framework — Always Do This Before Writing Code

1. UNDERSTAND: What exactly is being asked? Re-read until you can explain it in one sentence.
2. IDENTIFY INPUTS & OUTPUTS: What information do I have? What result am I producing?
3. PLAN IN PLAIN ENGLISH: Write the logic as if explaining to a friend. No code yet.
4. TRANSLATE to Java: Now convert your plain-English plan to actual code, one line at a time.

2.1 Teach Students to Ask the Right Questions

Before any if-else, students should ask:

- What am I checking? (the condition)
- What should happen if it's TRUE?
- What should happen if it's FALSE?
- Are there more than two possibilities? (else-if chain)

2.2 Plain-English First — Example Walkthrough

Problem: A store gives a 20% discount to customers who spend more than 500 EGP.

Step 1 — Plain English Plan:

IF the purchase amount is MORE THAN 500

→ apply a 20% discount

OTHERWISE

→ no discount, pay full price

Step 2 — Java Code:

```
double amount = 650;
double finalPrice;

if (amount > 500) {
    finalPrice = amount * 0.80; // 20% off
} else {
    finalPrice = amount;        // full price
}
```

Section 3 — If-Else Conditions

3.1 Basic if Statement

The if statement executes a block of code only when the condition is true. If it's false, nothing happens.

```
int temperature = 35;

if (temperature > 30) {
    System.out.println("It's hot! Drink water.");
}
// If temperature were 25, this block would be SKIPPED entirely
```

3.2 if-else Statement

Use else to define what happens when the condition is false — covering both paths.

```
int age = 16;

if (age >= 18) {
    System.out.println("You can vote.");
} else {
    System.out.println("You cannot vote yet.");
}
```

3.3 else-if Chain

When you have more than two possibilities, use else-if. Java checks each condition from top to bottom and runs the FIRST match only.

```
int score = 75;
```

```
if (score >= 90) {
    System.out.println("Grade: A");
} else if (score >= 80) {
    System.out.println("Grade: B");
} else if (score >= 70) {
    System.out.println("Grade: C");
} else if (score >= 60) {
    System.out.println("Grade: D");
} else {
    System.out.println("Grade: F");
}
```

Teaching Tip — Order Matters!

Ask students: What would happen if we put `score >= 60` FIRST?

A score of 95 would incorrectly get Grade D, because `95 >= 60` is true!

Always order conditions from MOST SPECIFIC (highest) to LEAST SPECIFIC (lowest).

3.4 Nested if-else

You can place an if-else inside another if-else when a decision depends on a previous one.

```
int age = 20;
boolean hasTicket = true;

if (age >= 18) {
    if (hasTicket) {
        System.out.println("Welcome! Enjoy the concert.");
    } else {
        System.out.println("You need a ticket to enter.");
    }
} else {
    System.out.println("Must be 18+ to attend.");
}
```

Best Practice — Don't over-nest!

More than 2-3 levels of nesting makes code hard to read. Often you can combine conditions:

`if (age >= 18 && hasTicket)` is cleaner than nested ifs for this case.

Section 4 — Practice Problems

Apply the 4-step framework to every problem. No typing code until Step 3 is done on paper!

Easy Problem — Odd or Even

Problem Statement:

Write a Java program that reads an integer number and prints whether it is Odd or Even.

Expected Output:

Input: 7 → Output: 7 is Odd
Input: 4 → Output: 4 is Even

Thinking Guide — Answer these before coding:

1. What mathematical property tells us if a number is even? (Hint: think about remainder)
2. Which operator gives you the remainder of a division?
3. Write the condition in plain English: 'A number is even when ...'
4. Now write it in Java!

Solution:

```
int number = 7;  
if (number % 2 == 0) {  
    System.out.println(number + " is Even");  
} else {  
    System.out.println(number + " is Odd");  
}
```

Key insight: `number % 2 == 0` means 'the remainder when divided by 2 is zero' = even.

● Medium Problem — BMI Calculator

Problem Statement:

Write a program that takes a person's weight (kg) and height (meters), calculates their BMI, and prints their weight category.

BMI Formula: $BMI = weight / (height * height)$

$BMI < 18.5$	→ Underweight
$18.5 \leq BMI < 25.0$	→ Normal weight
$25.0 \leq BMI < 30.0$	→ Overweight
$BMI \geq 30.0$	→ Obese

Thinking Guide — Work through these before coding:

5. What are the INPUTS? What is the OUTPUT?
6. Why should weight and height be double, not int?
7. How many categories are there? Do you need a simple if-else, or an else-if chain?
8. Write the full logic in plain English first. Then translate to Java.
9. TEST your solution: weight=70kg, height=1.75m → BMI ≈ 22.86 → Normal weight

Solution:

```
double weight = 70;  
double height = 1.75;  
double bmi = weight / (height * height);
```

```
System.out.printf("BMI: %.2f%n", bmi);

if (bmi < 18.5) {
    System.out.println("Underweight");
} else if (bmi < 25.0) {
    System.out.println("Normal weight");
} else if (bmi < 30.0) {
    System.out.println("Overweight");
} else {
    System.out.println("Obese");
}
```

Note: We don't need to check the lower bound in each else-if because Java already ruled it out in the previous condition.

Section 5 — Teaching Tips & Common Mistakes

Mistakes to Highlight in Class

- Using `=` instead of `==` in conditions — demo this live and show the compiler error
- Integer division: `7 / 2` gives 3, not 3.5 — always demo this in a live run
- Missing braces `{ }` with single-line if — works but causes bugs when code is added later
- Overlapping else-if conditions — show how order changes the result
- Forgetting that else-if chains stop at the first true condition

Engagement Strategies

- Before showing code — draw a flowchart on the board using the problem's logic
- Ask students to predict the output of a code snippet before running it
- Introduce 'tracing': step through code line by line on the board with a sample value
- Use real-world examples: ticket prices, grade calculators, temperature alerts
- Give intentionally broken code and ask students to find and fix the bug